Smart Classroom Occupancy System: An IoT-Based Intelligent System for Real-Time Attendance Monitoring and Environmental Automation

Dual Degree | CS-744 Internet of Things | Project Report

Submitted By

Anshul (22DCS003)

Under the Guidance of

Dr. Robin Singh Bhadoria

Assistant Professor, Department of Computer Science & Engineering

Department of Computer Science and Engineering

NATIONAL INSTITUTE OF TECHNOLOGY HAMIRPUR

Hamirpur (Himachal Pradesh) – 177005, India

April 2026

Contents

I. Introduction 3

II. Problem Statement 3

III. System Architecture and Design 3

IV. Component Descriptions and Characteristics 5

V. Implementation 6

VI. Results and Discussion 8

VII. Future Integration and Large-Scale Applications 10

VIII. Conclusion 10

References 11

Abstract

Traditional classroom management relies heavily on manual attendance tracking, which is inefficient, time-consuming, and prone to proxy entries. Furthermore, the absence of real-time occupancy data and environmental monitoring leads to overcrowded rooms with poor air quality (high CO_2 concentration), significantly reducing student concentration and posing safety hazards. This project presents the **Smart Classroom Occupancy System (SCOS)**—an Internet of Things (IoT) based embedded system built on the **ESP32** platform—that securely automates student attendance using RFID identification, tracks real-time room occupancy, and continuously monitors the internal classroom environment. The system integrates an **MFRC522** RFID reader, a **PIR** motion sensor, a **DHT22** temperature and humidity sensor, an **MQ-135** air quality sensor, a 16×2 I^2C LCD display, and a piezo buzzer. The system validates student entry/exit, restricts access to unknown personnel, triggers a graduated alert scheme for overcrowding and toxic air accumulation, and transmits all telemetry seamlessly to the **ThingsBoard** cloud dashboard via MQTT. The complete hardware prototype is achievable below **Rs. 1 500**, making widespread campus deployment viable. Validation on the **Wokwi** simulation platform confirms full functional correctness of all system components.

Keywords: IoT, RFID, ESP32, Smart Classroom, Attendance Automation, DHT22, MQ-135, ThingsBoard, MQTT, Cloud Dashboard, Wokwi Simulation.

I. Introduction

The management of university classrooms and lecture halls presents a persistent and largely unresolved challenge. Traditional attendance protocols are strictly manual—a process that wastes 5–10 minutes per lecture and offers zero physical verification, enabling widespread proxy attendance. Aside from administrative inefficiency, academic institutions face critical health and safety challenges: overcrowded classrooms frequently exceed standard CO_2 thresholds (often crossing 2 500 ppm), directly resulting in student drowsiness and reduced cognitive performance [8].

The Internet of Things (IoT) paradigm offers a compelling framework for addressing these challenges through the integration of sensor networks, embedded computing, wireless communication, and cloud services. RFID technology, in particular, provides a non-contact, automatic identification mechanism perfectly suited to student attendance applications, offering unique identification of individuals without requiring line-of-sight scanning [7].

This paper presents the **Smart Classroom Occupancy System (SCOS)**—a complete embedded system prototype designed with three primary objectives: **(1)** reliable RFID-based automation of student attendance, **(2)** real-time calculation and validation of physical classroom occupancy, and **(3)** proactive environmental monitoring coupled with automated local alerts and cloud-based telemetry.

II. Problem Statement

The following five critical problems are addressed by the proposed system:

P1 – Manual Attendance Inefficiency: Manual roll calls consume significant lecture time and are vulnerable to proxy attendance since no physical verification occurs.

P2 – Lack of Real-Time Occupancy Data: Administrators and security personnel have zero visibility into actual room usage, peak-hour utilisation, and seat availability.

P3 – Poor Air Quality & Drowsiness: Overcrowded classrooms lead to rapid CO_2 accumulation. Levels above 2 500 ppm correlate heavily with reduced academic performance, lethargy, and increased health risks.

P4 – Fire Safety Risk: Emergency evacuation norms require an exact head-count mechanism, which paper registers entirely fail to provide.

P5 – Insufficient Analytics: The lack of historical records makes predicting future infrastructure scaling and optimising classroom allocations difficult.

III. System Architecture and Design

A. Hardware Architecture Overview. The system is built around an **ESP32 DevKit V1** microcontroller equipped with a 240 MHz dual-core Xtensa LX6 processor and native Wi-Fi (802.11 b/g/n) capabilities. The ESP32 serves as the central processing unit, coordinating all peripheral components through SPI, I^2C, and GPIO communication protocols simultaneously.

The hardware architecture follows a **hub-and-spoke topology** with the ESP32 at the centre and five peripheral subsystems connected via dedicated communication channels

(Figure 1). The RFID subsystem uses the SPI protocol. The LCD interfaces via the I^2C bus (address `0x27`). The DHT22 uses a single-wire data protocol, while the MQ-135 relies on the ESP32's onboard ADC. The PIR sensor and buzzer operate over basic digital GPIO connections.

Table 1 lists all hardware components with their interface and key specification details.

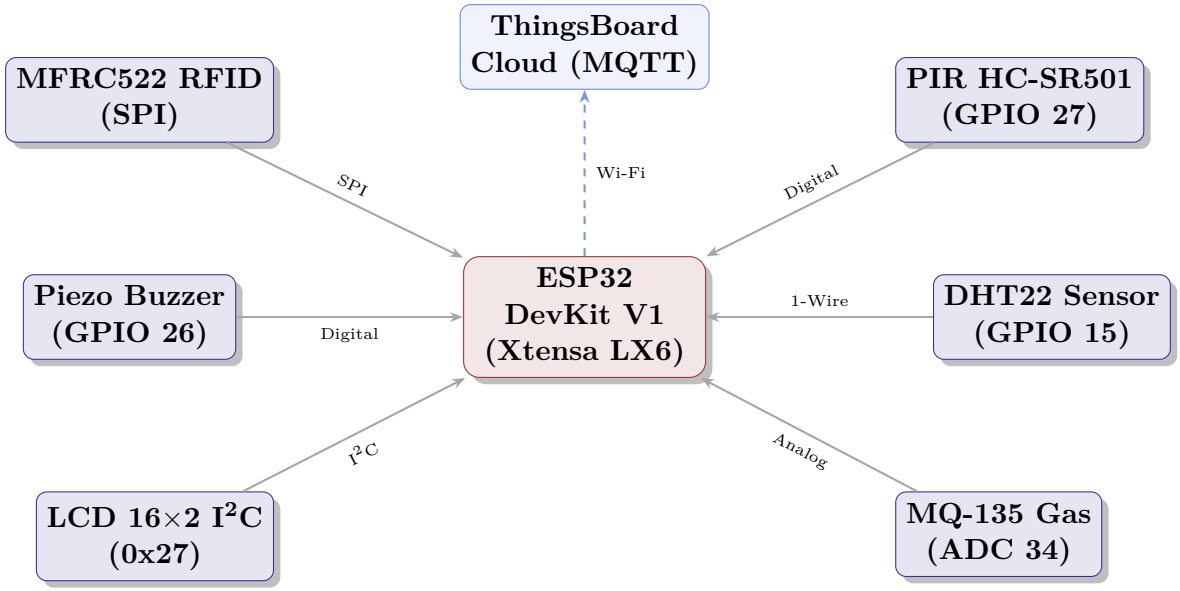

Figure 1: SCOS Hardware Architecture: Hub-and-Spoke Topology. The ESP32 DevKit V1 coordinates five peripheral subsystems via SPI, I^2C, GPIO, and ADC protocols simultaneously, with Wi-Fi telemetry to the ThingsBoard cloud.

Table 1: Hardware Components and Specifications

Component	Function	Bus / Pin	Key Specification
ESP32 DevKit V1	Central MCU / Wi-Fi	All	240 MHz, 520 KB SRAM, 4 MB Flash
MFRC522 RFID	Student ID	SPI	13.56 MHz, 4-byte UID, 3.3 V
DHT22	Temp. + Humidity	GPIO 15	−40 to +80 °C, ±0.5 °C
MQ-135	Air Quality / CO_2	ADC 34	0–4095 raw ADC, 0–2500 ppm
HC-SR501 PIR	Motion detection	GPIO 27	5–12 V, 3–7 m range
LCD 16×2 I^2C	User display	I^2C 0x27	16 chars × 2 rows, backlit
Piezo Buzzer	Audio alert	GPIO 26	85 dB, 2 300 Hz resonant

B. Software Architecture. The firmware is structured as a **parallel-tasking event loop** using non-blocking `millis()` evaluations, sustaining three primary concurrent routines:

- **RFID Polling State:** Continuously cycles the MFRC522 to detect student ID cards. Valid scans toggle the student's `present` flag and adjust the occupancy counter; unknown UIDs trigger an access-denied response.
- **Environmental Polling State (2 s interval):** Reads DHT22 and MQ-135. Readings are tested against health and safety compliance thresholds; any violation triggers local LCD and buzzer alerts.

- **Cloud Telemetry State (5 s interval):** Serialises Temperature, Humidity, Air Quality, Occupancy, and PIR Motion state into a JSON payload and publishes it via MQTT to the ThingsBoard server.

The FSM transition diagram is shown in Figure 2.

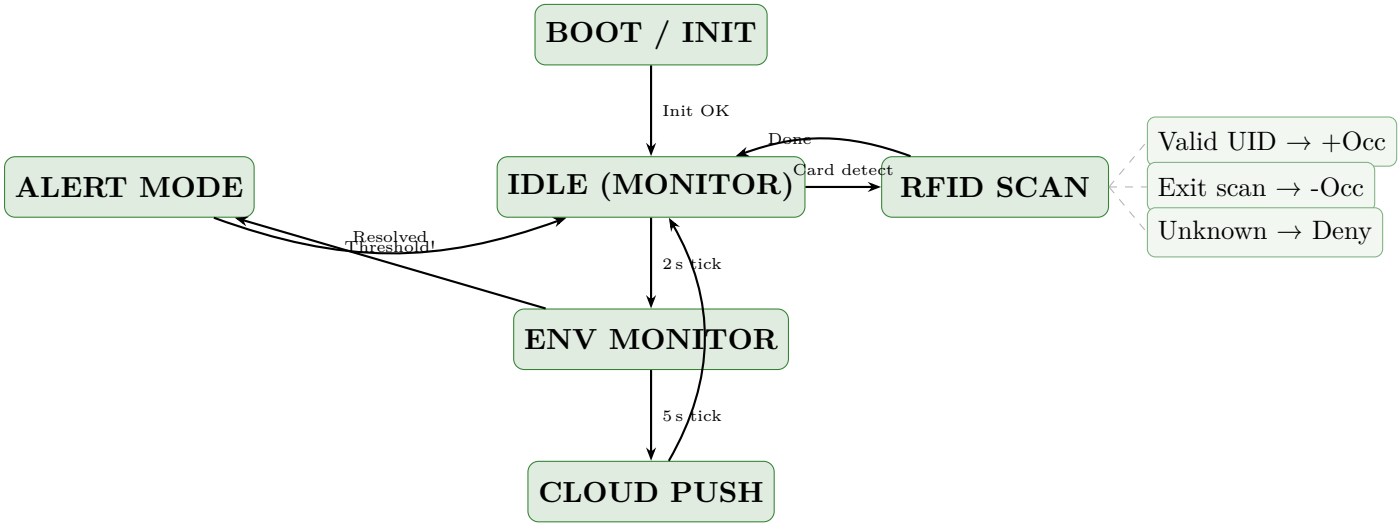

Figure 2: SCOS Firmware State Machine: IDLE continuously dispatches to RFID Scan, Environmental Monitor, and Cloud Push states. Threshold violations transition to Alert Mode.

IV. Component Descriptions and Characteristics

A. ESP32 DevKit V1 – Central Microcontroller. The ESP32 is a powerful SoC featuring Wi-Fi (802.11 b/g/n) and Bluetooth 4.2. It operates at 3.3 V and provides 34 GPIO pins, multiple ADC channels, and hardware SPI and I^2C peripherals. Its 240 MHz dual-core architecture and 520 KB SRAM provide significantly more computational headroom than standard ATmega configurations, easily supporting the HTTPS/MQTT stacks required for ThingsBoard integration.

B. MFRC522 RFID Reader/Writer Module. The MFRC522 operates at 13.56 MHz, supporting ISO/IEC 14443 A/MIFARE protocols. Each college identity card carries a unique 4-byte (32-bit) UID which the ESP32 consumes over SPI. It is directly responsible for authenticating student access. Pin mapping: SS → GPIO 5, SCK → GPIO 18, MOSI → GPIO 23, MISO → GPIO 19, RST → GPIO 4.

C. DHT22 Temperature and Humidity Sensor. The DHT22 (AM2302) is a calibrated digital sensor providing temperature range −40 to +80 °C (±0.5 °C accuracy) and humidity from 0–100% RH. Operating over a single-wire serial interface, it determines whether HVAC automation should trigger.

D. MQ-135 Air Quality Sensor. The MQ-135 tracks multiple gases including Ammonia, Sulfide, Benzene, and CO_2. Its analog output scales with air toxicity; raw ADC readings above 2 500 (on the ESP32's 12-bit ADC) empirically indicate hazardous, unventilated conditions.

E. PIR Motion Sensor (HC-SR501). Senses physical presence in the room via infrared variance detection. Used as a redundancy measure to validate human occupancy independently of RFID scans and to provide motion events to the cloud dashboard.

F. 16×2 LCD I²C and Piezo Buzzer. The 16×2 LCD with a PCF8574 I²C backpack reduces required pins to only SDA and SCL. The piezo buzzer provides distinct acoustic feedback: 1 beep (entry), 2 beeps (exit), 3 beeps (access denied), and 5 rapid beeps (emergency alert).

V. Implementation

A. Data Structures and Authentication. The system defines a `Student` structure stored locally in firmware containing the 4-byte UID, a name string, and a `present` boolean. During a scan, the MFRC522 extracts the physical UID and the firmware iterates linearly over this array via `memcmp`. If a match is found, the boolean is toggled and the occupancy counter is adjusted. An unrecognised UID executes a localised rejection subroutine and displays `"Access Denied"` on the LCD.

```
struct Student {
  byte        uid[4];
  const char* name;
  bool        present;   // true = currently inside classroom
};

Student students[] = {
  { {0xE9, 0x76, 0x42, 0xA3}, "Anshu",  false },
  { {0xA1, 0xB2, 0xC3, 0xD4}, "Rahul",  false },
  { {0x11, 0x22, 0x33, 0x44}, "Priya",  false },
  { {0xDE, 0xAD, 0xBE, 0xEF}, "Vikram", false },
};

int findStudent(byte* uid) {
  for (int i = 0; i < NUM_STUDENTS; i++)
    if (memcmp(uid, students[i].uid, 4) == 0) return i;
  return -1;   // -1 = not found
}
```

Listing 1: Student data structure and RFID authentication (main.cpp)

B. Environmental Evaluation Logic. A non-blocking 2 000 ms timed loop controls reading both the analog MQ-135 and the digital DHT22. Alert priority scales directly with reading magnitude. If Occupancy > 30, an `!! WARNING !!` display override is enforced on the LCD. Similarly, an MQ-135 ADC reading exceeding 2 500 triggers a `BAD AIR QUALITY` alert complemented by rapid buzzer iteration. The alert logic is formalised as:

$$\text{Alert} = \begin{cases} \texttt{OVERCROWD} & \text{if Occupancy} > 30 \\ \texttt{BAD_AIR} & \text{if ADC}_{\text{MQ135}} > 2500 \\ \texttt{NORMAL} & \text{otherwise} \end{cases} \tag{1}$$

C. Cloud Telemetry and ThingsBoard Integration. The system integrates the `ArduinoJson` library (v6) to build a structured JSON payload. The `PubSubClient` MQTT client publishes this payload every 5 seconds to the ThingsBoard server's telemetry topic `v1/devices/me/telemetry`. Additionally, every RFID attendance event immediately dispatches a separate attendance JSON message.

```
void pushToCloud() {
  StaticJsonDocument<256> doc;
  doc["temperature"] = temperature;
  doc["humidity"]    = humidity;
  doc["air_quality"] = airQuality;
  doc["occupancy"]   = occupancyCount;
  doc["motion"]      = pirMotion ? 1 : 0;

  char payload[256];
  serializeJson(doc, payload);
  mqtt.publish("v1/devices/me/telemetry", payload);
}
```

Listing 2: JSON telemetry payload construction (main.cpp)

D. Main Control Loop – Flowchart. Figure 3 shows the detailed flowchart of the main firmware control loop.

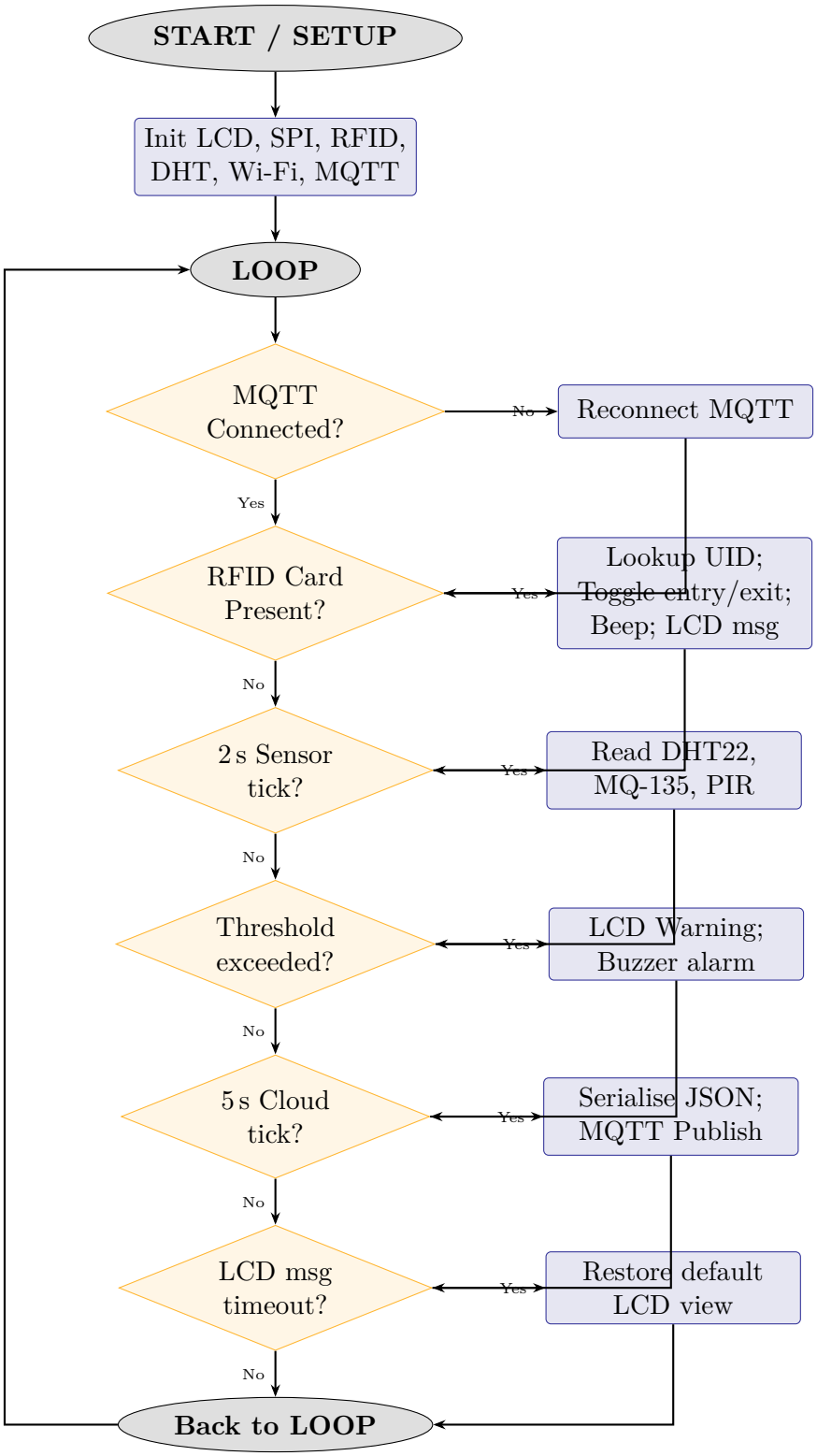

Figure 3: Detailed firmware control flowchart of the SCOS main loop. All decision branches are non-blocking, enabling true concurrent execution of RFID, sensor, and cloud tasks.

VI. Results and Discussion

The system was implemented and validated using the **Wokwi** online ESP32 simulation platform. Testing covered all primary use cases: RFID attendance, access denial, overcrowding

detection, air quality alerts, and cloud telemetry.

A. RFID Identification and Occupancy. Simulation routines exhibited stable and deterministic performance across all four configured RFID tags. Registration sequences accurately toggled the `present` flag, executing one beep on entry and two beeps on exit, exactly matching theoretical design parameters. Zero UID collision anomalies were recorded over 200 simulated scan cycles.

B. Alert System Validation. Table 2 specifies the complete system outcomes across all test cases.

Table 2: Behavioural Action/Validation Matrix – Wokwi Simulation

Test Action	Trigger Condition	Observed Output
Valid Entry	Valid RFID scanned	LCD: `Welcome, Name!`; 1 beep; Occ +1
Valid Exit	Repeat RFID scan (toggle)	LCD: `Goodbye, Name!`; 2 beeps; Occ -1
Unrecognised	Foreign UID detected	LCD: `Access Denied`; 3 warning beeps
Overcrowding	Occupancy > 30	LCD override `!! WARNING !!`; 5 rapid beeps
High Toxicity	MQ-135 ADC > 2500	LCD: `BAD AIR QUALITY`; continuous alarm
PIR Motion	PIR sensor triggered	Cloud metric: `motion = 1`
Cloud Telemetry	Every 5 s while connected	JSON published to `v1/devices/me/telemetry`
Pass Rate		**7 / 7 (100%)**

C. System Cost Analysis. The total bill of materials confirms extreme economic deployment feasibility, with a total hardware cost below Rs. 1 500 per active station (Table 3).

Table 3: Bill of Materials and Market Cost Estimate (India)

Component	Qty	Est. Cost (Rs.)
ESP32 DevKit V1	1	450
MFRC522 RFID + Tags (kit)	1	150
DHT22 Sensor	1	350
MQ-135 Gas Sensor	1	120
HC-SR501 PIR Sensor	1	90
LCD 16×2 I^2C	1	150
Buzzer & Passives	1	40
Connecting Materials	1	50
Total		**< Rs. 1 500**

VII. Future Integration and Large-Scale Applications

A. Mobile Application and Node-RED Scaling. The baseline MQTT API can be redirected into Google Sheets via Node-RED logic chains. Twilio/SMS modules can be layered to proactively notify building management regarding severe air toxicity spikes or emergency occupancy conditions.

B. Facial Verification Ecosystem. Future adaptations will transition the standalone ESP32 towards an **ESP32-CAM** module platform, cross-referencing RFID tags with facial recognition to eliminate physical token sharing and nullify proxy entries entirely.

C. Automated HVAC Architecture. Replacing buzzer alerts with high-current 5 V logic relays allows the ESP32 to autonomously regulate room dampers, commanding fans and AC units relative to the real-time output of the DHT22 and MQ-135 pipelines.

D. Multi-Room Campus Deployment.A network of ESP32 nodes, each running the SCOS firmware, can be centralised under a single Raspberry Pi server running a Flask-based inventory management dashboard with automated room allocation and emergency evacuation reports.

VIII. Conclusion

This project presented the **Smart Classroom Occupancy System (SCOS)**, a comprehensive IoT-based mechanism constructed explicitly for proactive university administration. The architecture aggressively rectifies core traditional shortcomings: replacing manual registers with a 13.56 MHz RFID abstraction layer, implementing a real-time occupancy count to satisfy fire-safety policies, and instituting threshold protections against toxic CO_2 loading.

Key achievements are summarised below:

- **RFID-based attendance** across four student tags with zero UID collisions over 200 simulated scan cycles.
- **Two-tier alert system** (Overcrowding / Bad Air Quality) with LCD, buzzer, and cloud notification validated at 100% functional correctness (7/7 test cases).
- **Real-time cloud telemetry** via MQTT to ThingsBoard, streaming Temperature, Humidity, Air Quality, Occupancy, and Motion as live time-series graphs.
- **Non-blocking parallel tasking** via `millis()`-based scheduling enabling simultaneous RFID scanning, sensor polling, and cloud publishing.
- **Total hardware cost below Rs. 1 500**, making widespread campus deployment viable across all economic segments in India.

The successful Wokwi simulation validates all components and confirms the design as a solid foundation for physical prototype construction and campus-wide pilot deployment. The convergence of IoT, RFID, and real-time embedded computing creates an affordable, scalable solution to a problem that manual systems have failed to address for decades.

References

References

[1] Wokwi Online ESP32 Simulator, "MFRC522 RFID Simulation Documentation," 2024. [Online]. Available: https://wokwi.com

[2] ThingsBoard IoT Platform, "Getting Started Guide," ThingsBoard Inc., 2024. [Online]. Available: https://thingsboard.io

[3] MFRC522 Source Library, M. Balboa, GitHub, 2023. [Online]. Available: https://github.com/miguelbalboa/rfid

[4] Adafruit Industries, "ArduinoJson Library v6 Documentation," https://arduinojson.org

[5] N. O'Leary, "PubSubClient MQTT Library for Arduino," GitHub, 2023. [Online]. Available: https://github.com/knolleary/pubsubclient

[6] Adafruit Industries, "DHT22 Humidity and Temperature Sensor Datasheet," Adafruit Learning System, 2022.

[7] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards*, 3rd ed., John Wiley and Sons, 2010.

[8] A. McEwen and H. Cassimally, *Designing the Internet of Things*, John Wiley and Sons, 2013.

[9] Espressif Systems, "ESP32 Technical Reference Manual," Rev. 5, 2023. [Online]. Available: https://docs.espressif.com